Where Does Your Data Live?

Week 1 — Pointers, Dynamic Memory, and Simple Contracts

Dr. Auqib Hamid Lone

PCC CS-301: Data Structures
GCET, Ganderbal

Week 1

Today's Three Questions

1. Where does a running C program keep its data?
2. How can we use memory that we request while the program runs?
3. How can we describe a data structure before implementing it?

Plan

First draw the memory. Then use one pointer. Then write one simple contract.

Motivation: Correct Is Not Always Fast Enough

A correct program can still be unusable

A loop that scans student records one by one always finds the right student — and with 5 records it is instant.

- With 10 million records and a query every millisecond, the *same* correct loop is far too slow.
- The answer is the same; the *arrangement* of the data is the difference.
- This course is the catalogue of arrangements and the judgement to choose between them [2, 1].

Data structures keep a correct program useful as the data grows.

Motivation: "It Felt Fast" Is Not an Argument

Sedgewick's opening question

"How long will my program take? Why does my program run out of memory?" — questions far too vague to answer precisely without a model [4].

- "It felt fast in my test" tells us nothing about tomorrow's input.
- Week 2 gives us the model: count the operations as a function of the amount of data, then compress the count into $O/\Omega/\Theta$.
- Week 1's job is the raw material that analysis counts — how data is laid out and reached in memory.

By the end of Week 12 you will say *why* a structure is fast, not just that it felt fast.

The Course in One Roadmap

1. **Weeks 1–2: Foundations** — memory, pointers, and the cost of work.
2. **Weeks 3–5: Linear structures** — arrays, stacks, queues, expression processing.
3. **Weeks 6–7: Linked structures** — singly/doubly linked lists, their applications.
4. **Weeks 8–9: Finding and ordering** — searching, elementary and efficient sorting.
5. **Week 10: Files and hashing** — file organization, hash tables.
6. **Weeks 11–12: Connected data** — trees, BSTs, AVL, graphs, MSTs, shortest paths.

Each week repeats one shape: *state the contract, then implement it, then analyse what it costs.*

One Running Thread Through the Whole Course

Our running example

A small expression processor reads `(a+b)*c` and must check its brackets, convert it, evaluate it, and look up its identifiers.

- A **stack** checks the brackets and drives the conversion (Weeks 3–4).
- A **symbol table** — first a list (Week 6), later a hash table (Week 10) — stores the identifiers.
- An **expression tree** gives the evaluation (Week 11).
- Every new structure arrives as the fix for something this program cannot yet do.

This is not twelve unrelated topics; it is one program learning new tricks each week.

What You Already Know (from prior courses)

- Variables store values.
- Arrays store several values in a row.
- Functions divide a program into manageable pieces.
- Loops repeat work.
- C gives you `struct` for grouping fields.

What this course adds

We ask: *what arrangement of data fits this job?* — and we answer with a contract, an implementation, and a cost.

Before any structure: where does a program even put its data?

A Variable Has a Place

Example

`int marks = 80;` stores the value 80 somewhere in memory.

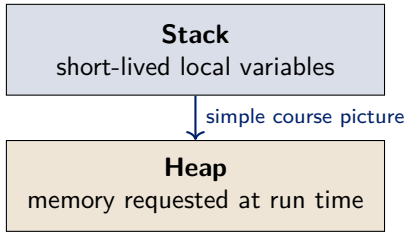
- The variable has a name: `marks`.
- It has a value: 80.
- It also has an address: its place in memory.

An address is like a house number. It tells us where to look.

Section 1

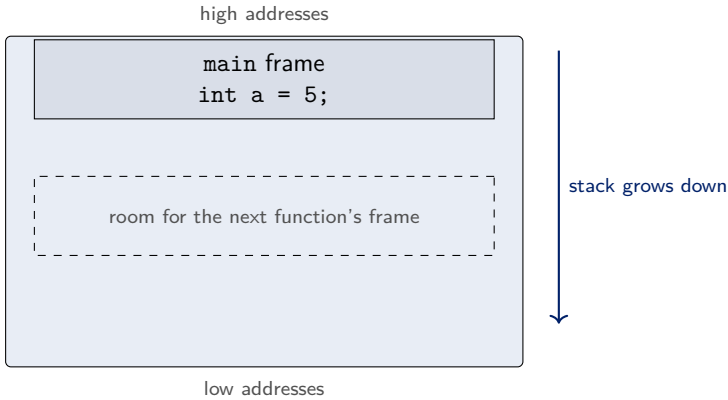
Memory

Two Places to Remember



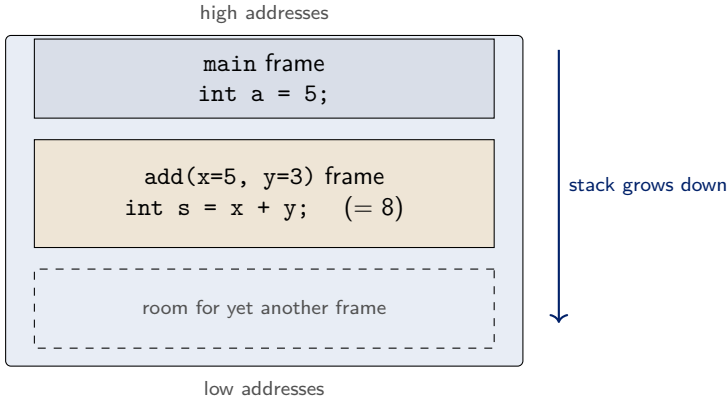
- Local variables usually live in the stack.
- Dynamic memory lives in the heap.
- Every later diagram in the course keeps this orientation: stack up, heap down.

How the Stack Grows — Step 1: `main` starts



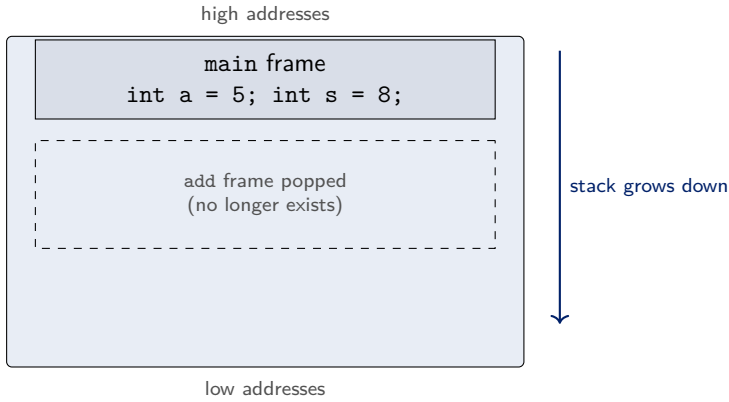
- Starting `main` creates one frame holding its local variables.
- Every new function call pushes a new frame just below the current one.

How the Stack Grows — Step 2: `add` is called



- The call to `add` pushed a second frame with its parameters and locals.
- The new frame sits at a **lower** address than `main`'s frame.

How the Stack Grows — Step 3: `add` returns



- Returning popped the frame, and its memory is available again.
- The answer 8 was copied back into `main`'s frame.

The Stack: Automatic Storage

Local variable

A variable declared inside a function is created when the function starts.

- It is available while that function is running.
- It disappears when the function returns.
- The program manages this basic lifetime automatically.

Example

`int score = 80;` inside a function is a local variable.

Why Do We Need the Heap?

A changing problem

The user enters the number of records only after the program starts.

- We may need space for 3 records today and 300 tomorrow.
- We may want the records to remain after a helper function returns.
- A structure that can vary in size cannot be given a fixed amount of storage at compile time [5].

The heap gives us memory whose size and lifetime we request at run time.

Wirth's argument, p.111: if the size is only known at run time, the storage must also be allocated at run time.

Check Your Prediction

Which variable can outlive the function that created it?

```
int score = 80;          or          int *p = malloc(sizeof(int));
```

- The local variable `score` belongs to its function call.
- The heap block remains until the program releases it with `free`.
- The pointer is only the handle; the block is the data storage.

Turn and talk

What must remain available if another function needs the record later?

The exact call for this — `malloc` and `sizeof` — comes in a few slides.

To use the heap, we need a handle on it — a pointer.

Section 2

Pointers

What Is a Pointer?

Definition

A pointer is a variable that stores an address [5].

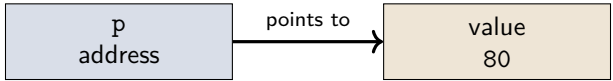
- An ordinary variable stores a value such as 80.
- A pointer stores a place such as "where the score lives".
- The type tells C what kind of value is at that place.

One declaration

`int *p;` means: `p` can point to an `int`.

Wirth (p.111): when a component's address must outlive the current scope, the compiler reserves fixed storage that holds the *address* of the component — not the component itself.

A Pointer Picture



- p does not contain the score itself.
- p tells us where the score can be found.

A Pointer Can Reach a Node

A two-field node

```
struct node {
  int data;
  struct node *next;
};
```

- `data` stores the value.
- `next` stores the address of another node.
- If there is no next node, set `next = NULL`.

This is the basic building block for a linked list in a later week. (The list handle will be called `head`; here the node is just `p`.)

The Safe Empty Pointer

`NULL`

`NULL` means that a pointer is not pointing to a valid object.

- Use it to show that a pointer is currently empty.
- Check a pointer before using the object it should point to.
- Never try to read through an empty pointer.

Picture

`struct node *p = NULL;` means “this pointer reaches nothing yet”.

Now let us actually request a block of heap memory and use it.

Section 3

One Heap Block

Requesting One Integer

The basic pattern

```
int *p = malloc(sizeof(int));
if (p == NULL) return 1;
*p = 80;
```

- `malloc` requests bytes from the heap.
- `sizeof(int)` asks C for the right number of bytes.
- `p` stores the address of the requested space.
- Check the result against `NULL` before using it.

Allocate a Complete Node

A safer engineering pattern

```
struct node *p = malloc(sizeof(struct node));
if (p == NULL) return 1;
p->data = 42;
p->next = NULL;
```

- `sizeof(struct node)` asks for exactly one node's bytes.
- `p->data` means: the `data` field of the node `p` points at.
- Initialising every field makes the node's state predictable.
- This node has a clear end: `p->next == NULL`.

Audit the Node Before You Ship It

Review checklist

A small program is not finished when it compiles.

1. Was the allocation checked against `NULL`?
2. Was every field initialised before the node was used?
3. Is there exactly one clear owner responsible for `free(p)`?
4. Can the program reach the node later, or has its pointer been lost?

Engineering habit

Test the normal path and the allocation-failure path.

Trace It Before Running It

After these lines, what is true?

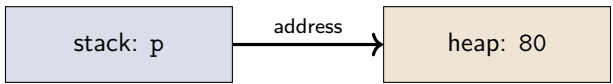
```
int *p = malloc(sizeof(int)); *p = 80;
```

1. `p` contains an address.
2. The heap block at that address contains 80.
3. The name `p` itself is not the heap block.

Challenge

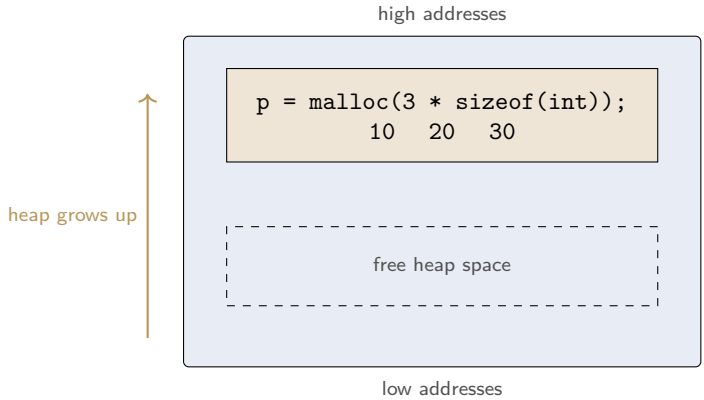
Which line would fail if `p == NULL`?

What Happened?



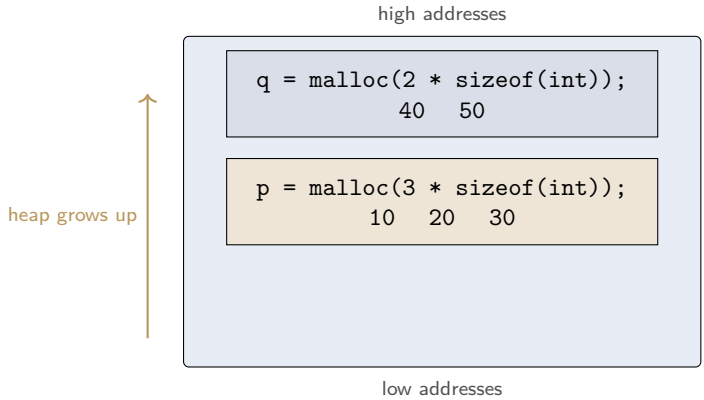
1. C found a free heap block.
2. The address was stored in `p`.
3. We used `*p` to write the value 80 there.

How the Heap Grows — Step 1: a block of three ints



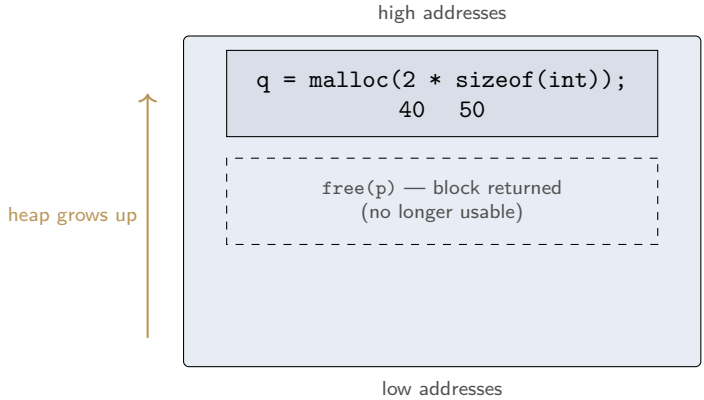
- One `malloc` with room for three integers **grew the used heap** upward.
- The block survives after the allocating function returns.

How the Heap Grows — Step 2: a second block



- The second request placed q's block at a **higher** address.
- Blocks are independent; each keeps its own data.

How the Heap Grows — Step 3: `free` returns a block



- `free(p)` gives `p`'s block back to the system.
- `q`'s block is untouched, and the used heap can shrink again.

Returning the Memory

When the block is no longer needed

```
free(p);
p = NULL;
```

- `free(p)` gives the heap block back to the system.
- Setting `p` to `NULL` makes the old pointer visibly empty.
- A block requested with `malloc` should eventually be freed.

Request memory, use it, return it.

Two Beginner Mistakes

- Memory leak: forget to call `free`; the block stays reserved.
- Dangling pointer: use a pointer after its block was freed.

Simple habit

Keep track of who requested the memory and who will return it.

Memory and pointers are the machinery. The contract is the map.

Section 4

Simple Contracts

What Is an ADT?

Abstract Data Type

An ADT describes what we can do with data, without saying how it is stored.

- Sedgewick (p.64): a data type is "a set of values and a set of operations on those values" [4].
- Karumanchi (p.18, PDF): an ADT has two parts — a declaration of data and a declaration of operations [3].
- The implementation is the code and memory arrangement behind the promise.

The two framings agree: "set of values and operations" is exactly "declaration of data and declaration of operations."

This Course Is a Tour of ADTs

The pattern you will see every week

Linked Lists, Stacks, Queues, Priority Queues, Binary Trees, Disjoint Sets, Hash Tables, and Graphs are all commonly used ADTs [3].

- Each one starts the same way: state the contract, *then* implement it.
- Weeks 3–12 work through this list one ADT at a time.
- Today's "bag" is a warm-up for that pattern, not a one-off example.

Watch for this shape: "What is a Stack?" then "Stack ADT" then "Implementation" — every week, same order.

Example: A Very Small Bag

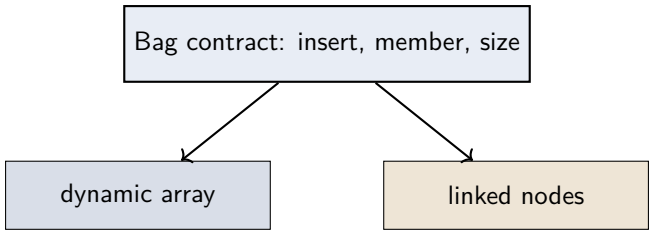
The contract

A bag of integers supports three operations:

1. `insert(x)` — put `x` into the bag.
2. `member(x)` — answer whether `x` is present.
3. `size()` — report how many items are present.

The contract does not yet say whether the bag uses an array or a list. A bag is one of Sedgewick's three collection types (p.120) — the other two (queue, stack) simply change which object is removed next [4].

One Contract, Two Possible Implementations



The key distinction

Same operations, different internal arrangement — the caller's code does not change when the bottom box does.

Contract or Implementation?

Classify each statement

Is it part of the contract, or part of one implementation?

1. "`member(7)` answers yes or no."
2. "The values are stored in an array."
3. "`size()` reports the number of values."

The first and third describe what users can ask (contract). The second describes how (implementation). State the contract before writing a line of the implementation.

Week 1 Summary

- **Memory:** the stack holds short-lived local variables; the heap holds memory requested while the program runs.
- **Pointers:** a pointer stores an address; `NULL` means "points to nothing."
- **Dynamic memory:** request with `malloc`, check the result, use the block, and call `free`.
- **ADTs:** describe operations first (a contract); a data structure is the implementation that honours it.
- The rest of this course is a tour of ADTs: contract first, implementation second, cost analysis third.

Next week

We learn how to talk about the amount of work an operation needs — $T(n)$, $O/\Omega/\Theta$, and best/worst/average case.

References I

Alfred V. Aho, John E. Hopcroft, and Jeffrey D. Ullman.
Data Structures and Algorithms.
Addison-Wesley, 1st edition, 1983.

Ellis Horowitz and Sartaj Sahni.
Fundamentals of Data Structures.
Galgotia Booksource, 2nd edition, 2008.

Narasimha Karumanchi.
Data Structures And Algorithms Made Easy.
CareerMonk Publications, 2017.
This calibre-generated edition carries no printed folio numbers; page numbers cited in CS301 material are PDF page indices. NOT a source for Week 1's C-specific pointer/malloc content — see this course's supporting-books index for the coverage check.

Robert Sedgewick and Kevin Wayne.
Algorithms.
Addison-Wesley, 4th edition, 2011.

References II

Niklaus Wirth.

Algorithms and Data Structures.

Author (freely distributed), 2004.

Oberon version, August 2004; revision of the 1985 Prentice-Hall edition. Page numbers cited in CS301 material refer to this Oberon PDF, not to the 1985 print edition.